SPRINT 3 COMPLETION REPORT

SHADOW OS v2 — Memory Foundation: MemoryManager

' COMPLETE — 396/396 TESTS

Date: 2026-07-07 · Sprint 3 of 5 · FOREX ENGINE PRO

Executive Summary

Sprint 3 delivers the **MemoryManager (MM)** — the append-first memory layer that gives the system permanent, versioned, crash-proof recall of everything it has ever observed. Nothing lives in-process: every memory is written to PostgreSQL the moment it is created, every change is a full-row snapshot in an append-only history table, and history is never deleted. A memory can be appended to, re-scored, tagged, archived and restored — but its original payload, event type and timestamp are immutable forever.

The original Sprint 3 plan ("TTL-based memory_entries") was upgraded during design review to a **two-table architecture**: the pre-existing memory_entries table is retained as a KV/TTL working cache, while the new memory_events + memory_event_history pair provides the permanent event memory the Sacred Constraint demands. TTL expiry never touches event memory.

Deliverables

1. Migration 004 (telemetry/migrations/004_memory_foundation.sql)

- memory_events table — the permanent memory store (append-first, versioned, status-tracked)
- memory_event_history table — append-only audit trail; one full-row snapshot per change (SACRED: never deleted)
- memory_entries untouched — retained as KV/TTL cache (namespace-isolated, GC-managed)
- Partial unique index on dedupe_key (idempotent creates), GIN index on tags, B-tree indexes on domain/trade/strategy/time
- Applied cleanly: 0 data loss, all 14 tables verified, 29 events + 1 shadowm_trade preserved

2. MemoryManager (telemetry/managers/MemoryManager.js)

Implements the full 13-method Sprint 3 spec plus the KV cache surface.

Status machine:

```
ACTIVE    !’ ARCHIVED (archiveMemory)
ARCHIVED  !’ ACTIVE   (restoreMemory)
ACTIVE    !’ CORRUPTED (validateMemory — structural damage detected)
ARCHIVED  !’ CORRUPTED (validateMemory — archived rows are validated too)
CORRUPTED !’ ACTIVE   (restoreMemory — after manual repair)
```

Public API (spec methods):

Method	Description
createMemory()	New ACTIVE memory, version=1; idempotent on dedupe_key
appendMemory()	The ONLY way to add information — appends to context, never mutates payload
updateMemory()	Mutable fields only (importance/tags/reasoning/metadata); immutables rejected
archiveMemory()	ACTIVE !’ ARCHIVED; row remains fully readable
restoreMemory()	ARCHIVED/CORRUPTED !’ ACTIVE
searchMemory()	Filter by type/domain/strategy/symbol/tags/importance/text/time/status
queryByDomain()	All memories for a runtime domain
queryByTrade()	Full memory trail of one trade intent
queryByTime()	Time-window queries

queryByStrategy()	All memories for a strategy
tagMemory()	Add/remove tags with dedupe
summarizeMemory()	Aggregate counts, importance stats, time range, top tags — feeds snapshots
validateMemory()	Structural + referential + version-gap checks; quarantines to CORRUPTED

Additional surface:

Method	Description
getMemory() / getMemoryHistory()	Direct reads
kvSet() / kvGet() / kvGetAll() / kvGc()	KV/TTL cache on memory_entries
getStats() / ping() / init() / shutdown()	Operational surface

Safety properties:

- **Append-first:** payload, event_type, occurred_at, runtime_domain, trade_intent_id, strategy_id, symbol, source, dedupe_key are immutable forever — enforced in code
- **Atomicity:** SELECT FOR UPDATE + UPDATE + history INSERT in one transaction
- **Invariant:** history row count === version, for every memory, always
- **Idempotency:** ON CONFLICT (dedupe_key) WHERE dedupe_key IS NOT NULL DO NOTHING
- **Quarantine, never delete:** corruption marks rows CORRUPTED; no MM method deletes from memory_events or memory_event_history
- **KV isolation:** kvGc() can only touch memory_entries; event memory is physically separate

3. RDM Snapshot Integration (additive)

RuntimeDomainManager.takeSnapshot(reason, { memorySummary }) now stores a provided summarizeMemory() result in system_snapshots.memory_summary (placeholder when absent).

Backwards compatible — all 107 Sprint 1 tests pass unchanged.

4. Managers Barrel (telemetry/managers/index.js)

Exports RuntimeDomainManager, TradeIntentManager and MemoryManager.

5. Migration Runner Update (telemetry/migrations/run.js)

- Migration 004 added to run list
- memory_events, memory_event_history added to EXPECTED_TABLES (14 total)

Test Results

Sprint 3 Tests (101 total)

Suite	Tests	Pass	Fail
Unit (MemoryManager)	60	60	0
Integration (MM lifecycle & concurrency)	11	11	0
Integration (MM × RDM × TIM)	7	7	0
Simulation (persistence & crash)	9	9	0

Stress (concurrent load)	14	14	0
TOTAL	**101**	**101**	**0**

Full Regression

Sprint	Suite	Tests	Pass	Fail
0	Schema + smoke	19	19	0
1	RDM (unit/integration/sim/stress)	107	107	0
2	TIM (unit/integration/sim/stress)	169	169	0
3	MM (all suites)	101	101	0
TOTAL		**396**	**396**	**0**

Combined: 396/396 tests passing. Zero regressions.

Test Coverage Highlights

Unit Tests (60)

- Constructor guards, exported constants, init() table verification, pre-init guards, ping()
- createMemory: defaults, full-field create, CREATE history snapshot, dedupe idempotency, explicit occurred_at, 7 validation rejection classes
- appendMemory: context growth, version bump, calledBy attribution, immutability of payload/occurred_at, non-ACTIVE rejection
- updateMemory: all mutable fields, **all 9 immutable fields individually rejected** , unknown-field rejection, null reasoning
- tagMemory: add/remove/dedupe, audit detail, non-ACTIVE rejection
- archive/restore: round trip, double-archive rejection, restore-of-ACTIVE rejection, CORRUPTED!'ACTIVE path, archived rows fully readable
- search/queries: every filter, tagsAny (&&) vs tagsAll (@>), status filter incl. ANY, pagination, ordering, queryBy* delegation, real trade-intent linkage
- summarizeMemory: aggregate correctness, since filter
- validateMemory: clean pass, structural corruption !' CORRUPTED + audit, consistency_log entries, orphan WARN, version-gap WARN, markCorrupted=false dry-run
- KV: round trip, upsert, TTL expiry, GC, namespace isolation, **kvGc never touches memory_events**

Integration Tests (18)

- Full lifecycle chain create!'append×3!'update!'tag!'archive!'restore: 8 versions, 8 history rows, strict op sequence, monotonic versions
- Historical state reconstruction from snapshots
- 3× archive/restore cycles preserving all context layers
- Concurrency: 10 parallel appends (zero lost), 8-way dedupe race (1 row), archive-vs-update race (invariant holds), mixed ops across 5 memories
- Trading-day reconstruction via domain/strategy/time/tag/text queries
- KV churn coexistence; KV caching of derived summaries
- MM × TIM: intent lifecycle !' memory trail via queryByTrade; orphan detection

- MM × RDM: validateMemory !’ rdm.logConsistency; direct-INSERT fallback without RDM
- summarizeMemory !’ takeSnapshot !’ system_snapshots.memory_summary verified end-to-end
- Full pipeline: signal !’ intent !’ execution !’ memory !’ summary !’ snapshot

Simulation Tests (9)

- SIM-1: Restart — new pool + manager sees all memories/history/KV; continues mutating (2 tests)
- SIM-2: pg_terminate_backend mid-transaction — clean rollback, no partial state (1 test)
- SIM-3: Atomicity — forced history failure rolls back row update; history==version after mixed load (2 tests)
- SIM-4: External raw-SQL corruption — quarantined, salvageable data intact, repair+restore path (2 tests)
- SIM-5: Idle-connection massacre — pool recovers transparently (1 test)
- SIM-6: 3 simulated trading days across process restarts — full recall, zero drift (1 test)

Stress Tests (14)

- STRESS-1: 100 sequential creates (~5ms avg)
- STRESS-2: 50 concurrent creates; 100 concurrent creates over 50 dedupe keys !’ exactly 50 rows
- STRESS-3: 30 hot-row concurrent appends (zero lost); 30 mixed mutations, invariant holds
- STRESS-4: 50 concurrent searches; 30 concurrent summarize/stats/validate
- STRESS-5: 10KB payload; 100-append context, reads stay <500ms
- STRESS-6: GIN tag search <300ms; window search <300ms; summarize <1000ms
- STRESS-7: 100 concurrent KV upserts; 100 concurrent KV reads

Architecture Decisions

Two-Table Memory Split (design upgrade)

The original blueprint assigned MemoryManager to the TTL-based memory_entries table alone. A TTL cache cannot satisfy "memory survives crash and is never lost" — expiry is deletion. Sprint 3 therefore splits the memory layer:

- **memory_events** — permanent, append-first event memory (the system's experience)
- **memory_event_history** — full-row snapshots per change (the audit trail)
- **memory_entries** — unchanged, KV/TTL working cache for ephemeral state (cooldowns, market state)

kvGc() operates only on memory_entries by construction; the permanent tables have no delete path in any MM method.

Append-First over Update-In-Place

appendMemory() is the only way to add information to an existing memory. New facts land in the context JSONB array (context = context || $addendum), each stamped with appended_at/appended_by. The original observation is never rewritten — the system's past cannot be edited, only annotated.

CAS Pool Deadlock Prevention (found by stress test, fixed)

The dedupe path of createMemory() originally called this._pool.query() while still holding a pool client — the exact deadlock pattern documented in replit.md. Sequential and low-concurrency tests passed; STRESS-2 (100 concurrent creates, pool.max=5) deadlocked permanently. Fixed by reusing the already-held client for the duplicate lookup. This validates the stress-gate policy: the bug was invisible to every other suite.

Validation Severity Policy

- **Structural damage** (payload/context/metadata/tags wrong JSONB type, empty event_type) !' ERROR, row quarantined to CORRUPTED
- **Referential orphans** (trade_intent_id pointing nowhere) and **version/history gaps** !' WARN only, logged to consistency_log, row stays ACTIVE

All findings flow through rdm.logConsistency() when RDM is present, direct consistency_log INSERT otherwise.

Sacred Constraint Compliance

- 0 rows deleted from events (29 !' 29)
- 0 rows deleted from shadowm_trades (1 !' 1)
- memory_events and memory_event_history have no DELETE path in any MM method
- Corruption quarantines (CORRUPTED status) — never deletes
- TTL/GC confined to memory_entries cache by construction
- index.js and server.js are unchanged
- Migration 004 is additive-only and idempotent

Sprint 4 Readiness

Sprint 4 (KnowledgeManager) can begin. The memory layer now provides the raw experience stream that knowledge distillation will consume: searchMemory()/queryBy*() for training-data selection, summarizeMemory() for system-state snapshots, and memory_event_history for full replay.

Run all Sprint 3 tests:

```
node --test --test-reporter=spec \
  telemetry/tests/unit/MemoryManager.test.js \
  telemetry/tests/integration/mm_integration.test.js \
  telemetry/tests/integration/mm_rdm_tim_integration.test.js \
  telemetry/tests/simulation/mm_persistence.test.js
node --test --test-reporter=spec telemetry/tests/stress/mm_stress.test.js
```

(Stress suite runs separately — SIM-5 terminates idle DB connections and would disturb parallel test processes.)